

Construction Planning System

AI-Powered Construction Project Planning with IBM Granite 3.3 2B

Project Description

Construction Planning System is an intelligent project planning platform that leverages IBM's Granite 3.3 2B AI model (via Ollama) to provide comprehensive construction project analysis and planning recommendations. The platform addresses the challenge of accurate construction estimation by delivering AI-powered insights, detailed blueprints, resource allocation, cost analysis, and week-by-week construction schedules.

Using Granite 3.3 2B's advanced language capabilities, the system analyzes project parameters (built-up area, number of floors, timeline constraints), generates accurate worker requirements, cost breakdowns, material estimations, and detailed architectural blueprints tailored to individual project specifications. The system ensures fast response times through local Ollama deployment while maintaining high-quality planning advice through sophisticated calculation algorithms and AI-powered analysis.

Construction Planning System transforms project planning into an intelligent, user-friendly experience through its modern interface, comprehensive feature set, and AI-powered analysis that provides personalized construction guidance considering location-specific rates, building complexity, and realistic timelines.

Scenarios

Scenario 1:

A contractor wants to plan a residential G+2 (3-floor) building of 1000 sq. yards. The system detects the project parameters, analyzes building complexity, and provides:

- Worker requirements breakdown (masons, helpers, steel workers, carpenters, supervisors)
- Total labour days calculation
- Timeline estimation (days, weeks, months)
- Detailed cost breakdown (labor, materials, overhead)
- Material requirements (steel, cement, sand, water)
- Professional architectural blueprint with room layouts and dimensions

Scenario 2: Custom Timeline Planning

A developer has budget constraints and wants to complete a 1500 sq. yard project in 90 days. The system:

- Calculates required workforce for accelerated timeline
- Adjusts material quantities based on faster execution
- Provides cost implications of timeline compression
- Generates optimized weekly construction schedule
- Shows resource intensity requirements

Scenario 3: Cost Analysis & Budget Planning

A project manager needs to understand cost structure for a 2000 sq. yard, G+3 building. The system delivers:

- Detailed cost breakdown (labor, materials, overhead)
- Cost per sq. yard analysis
- Material cost estimates with current rates
- Labour cost projections
- Overhead and contingency calculations
- Customizable wage rates and material costs

Scenario 4: Detailed Construction Schedule

A contractor needs a comprehensive week-by-week plan including all finishing works. The system generates:

- Foundation to handover timeline
- All construction phases from site prep to final finishing
- Electrical, plumbing, painting, and finishing schedules
- Activity dependencies and durations
- Resource allocation per phase

Architecture Overview

Construction Planning System is built as a modular platform combining Flask backend with IBM Granite 3.3 2B AI model for intelligent project planning. The architecture prioritizes accuracy, speed, and user experience by leveraging local AI inference and sophisticated calculation algorithms.

Core Technologies

- **Flask:** Lightweight Python web framework for routing and request processing
- **IBM Granite 3.3 2B:** Local AI inference via Ollama for intelligent text generation
- **Ollama:** Local LLM runtime for Granite model execution
- **Python Math & Algorithms:** Construction calculation logic
- **HTML5/CSS3/JavaScript:** Modern, responsive frontend with smooth animations
- **Real-time Visualization:** Blueprint generation, timeline visualization
- **Local Deployment:** No cloud API keys required

Pre-requisites

Software Requirements

- **Python 3.8+:** Download from python.org
- **Ollama:** Download from ollama.ai
- **Granite 3.3 2B Model:** Pre-downloaded via Ollama (`ollama pull granite:3.3-2b`)
- **Git:** Download from git-scm.com
- **Code Editor:** VS Code, PyCharm, or any preferred IDE

Knowledge Prerequisites

- **Python Basics:** Functions, classes, OOP, exception handling
- **Flask Framework:** Routing, request handling, JSON responses
- **HTML/CSS:** Responsive design, Flexbox/Grid layouts
- **JavaScript:** DOM manipulation, Fetch API, async/await
- **Construction Domain:** Basic understanding of building terminology
- **Mathematics:** Area calculations, cost analysis, resource planning

Hardware Requirements

- **Processor:** Intel i5/AMD Ryzen 5 or better
- **RAM:** Minimum 8GB (16GB recommended for Ollama)
- **Storage:** 10GB free space (for Granite model)
- **Internet:** Required for initial setup only

Project Workflow

Phase 1: Environment Setup & Ollama Configuration (Week 1)

Establish local AI infrastructure and validate model connectivity

Activity 1.1: Set up Python Environment

- Create project directory structure
- Initialize virtual environment
- Create requirements.txt with dependencies
- Install all Python packages

Activity 1.2: Configure Ollama & Granite Model

- Download and install Ollama
- Pull Granite 3.3 2B model
- Verify model accessibility
- Test local inference capabilities

Activity 1.3: Validate System Configuration

- Test Flask application startup
- Verify Ollama API connectivity
- Test model response times
- Document configuration steps

Phase 2: Core Backend Development (Weeks 2-3)

Build Flask infrastructure and construction calculation logic

Activity 2.1: Set up Flask Application Structure

- Initialize Flask app with proper architecture
- Create blueprint routes structure
- Implement error handling middleware
- Set up request/response handling

Activity 2.2: Implement Construction Calculator

- Build worker calculation logic
- Implement timeline estimation algorithms
- Create cost calculation methods
- Develop material requirement estimations

Activity 2.3: Integrate Ollama/Granite API

- Create Ollama API communication module
- Implement prompt engineering for project analysis
- Add response parsing and validation
- Handle API timeouts and errors

Activity 2.4: Develop Blueprint Generation Engine

- Create room layout calculation system
- Implement dimension calculation based on area
- Build floor plan structure generation
- Add architectural visualization data

Activity 2.5: Build Weekly Schedule Generator

- Create construction phase definitions
- Implement timeline dependency logic
- Add resource allocation per phase
- Generate detailed activity breakdowns

Phase 3: Frontend Development (Weeks 4-5)

Create modern, responsive user interface

Activity 3.1: Design Responsive HTML Structure

- Create semantic HTML5 layout
- Design form sections for input
- Build results display templates
- Implement blueprint visualization container

Activity 3.2: Develop CSS Styling & Animations

- Implement modern color scheme
- Create responsive grid layouts
- Add smooth transitions and animations
- Build blueprint blue-print themed styling
- Develop mobile-responsive design

Activity 3.3: Create Interactive JavaScript Logic

- Implement form validation
- Build real-time calculation display
- Create blueprint visualization rendering
- Add timeline animation
- Implement export functionality

Activity 3.4: Build Results Display Components

- Create dynamic result cards
- Implement data visualization
- Build cost breakdown display
- Create material requirements display
- Implement assumption notes display

Phase 4: Integration & Testing (Week 6)

Integrate all components and comprehensive testing

Activity 4.1: End-to-End Integration

- Connect frontend to backend
- Test API communication
- Validate data flow
- Test error handling

Activity 4.2: Functional Testing

- Test with various project sizes
- Validate calculations accuracy
- Test different floor configurations
- Verify blueprint generation

Activity 4.3: Performance Testing

- Test response times
- Optimize Ollama queries
- Profile database queries
- Load testing

Phase 5: Deployment & Documentation (Week 7)

Deploy and document the system

Activity 5.1: Local Deployment

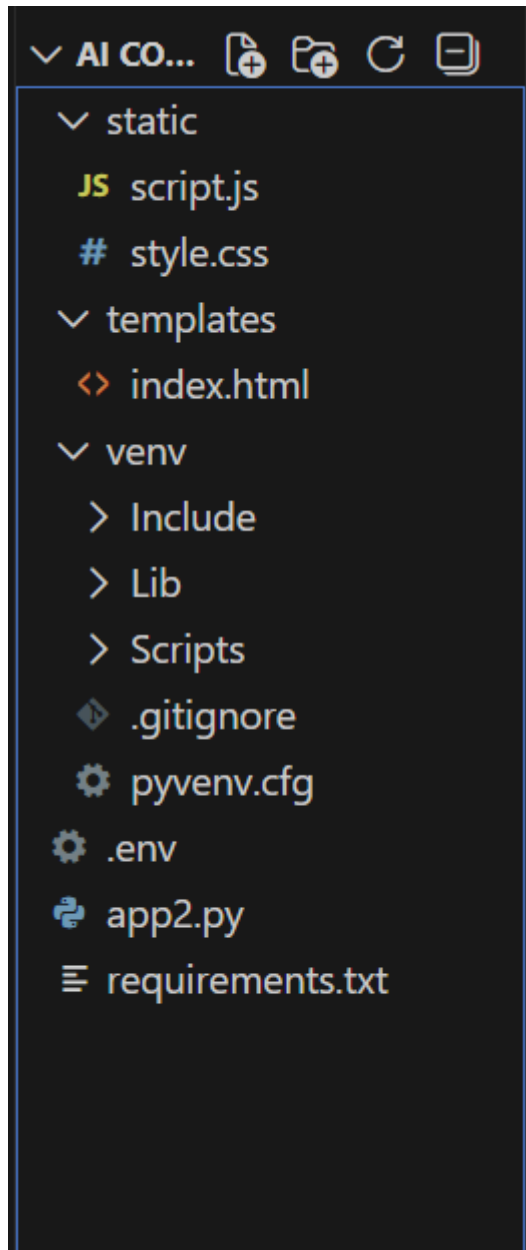
- Set up production environment
- Configure logging
- Document deployment steps
- Create user guide

Activity 5.2: Create Comprehensive Documentation

- Write API documentation
- Create user manual
- Document configuration
- Create troubleshooting guide

Technical Architecture

Project Structure:



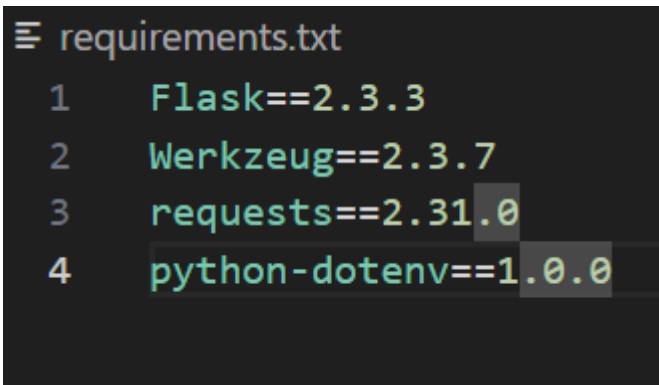
MILESTONE 1: Environment Setup & Ollama Configuration

Establish local AI infrastructure and validate model connectivity

This milestone focuses on setting up the complete local development environment including Python, Flask, and Ollama with the IBM Granite 3.3 2B model. It ensures that the AI model is correctly installed, verified, and ready for local inference.

Activity 1.1: Install Dependencies & Setup Environment

Create requirements.txt:



```
requirements.txt
1  Flask==2.3.3
2  Werkzeug==2.3.7
3  requests==2.31.0
4  python-dotenv==1.0.0
```

```
pip install -r requirements.txt
```

Activity 1.2: Configure & Verify Ollama Setup

Install Ollama:

1. Download Ollama from ollama.ai
2. Follow installation instructions for your OS
3. Start Ollama service

Pull Granite Model:

```
ollama pull granite:3.3-2b
```

Verify Installation:

```
ollama list
# Output should show: granite:3.3-2b
```

MILESTONE 2: Core Backend Development

This phase involves building the Flask backend, implementing construction calculation algorithms, and integrating the Ollama-Granite AI model. It enables worker estimation, cost calculation, material planning, and blueprint generation logic.

Activity 2.1: Flask Application Setup

Create app.py:

```
from flask import Flask, render_template, request, jsonify
import json
import time
import math

app = Flask(__name__)

# =====
# CONFIGURATION
# =====

MODEL_ID = "granite3.3:2b"

print("=" * 70)
print("🏗️ Construction Planning System")
print("🤖 Model:", MODEL_ID)
print("📍 Running locally via Ollama")
print("=" * 70)

# =====
# CALCULATION LOGIC
# =====

class ConstructionCalculator:
    """Calculate construction requirements based on project parameters"""

    # =====
    # CALCULATION LOGIC
    # =====

    # Standard rates for India (INR)
    DAILY_WAGE_PER_WORKER = 500 # ₹500 per day
    COST_PER_SQ_YARD = 1500 # ₹1500 per sq yard
    OVERHEAD_PERCENTAGE = 10 # 10% overhead

    # Material rates per unit (INR)
    STEEL_RATE_PER_TON = 60000 # ₹60,000 per ton
    CEMENT_RATE_PER_BAG = 420 # ₹420 per 50kg bag
    SAND_RATE_PER_TON = 800 # ₹800 per ton

    # Standard thumb rules
    STEEL_PER_SQ_YARD = 3.5 # kg per sq yard (for RCC)
    CEMENT_PER_SQ_YARD = 0.4 # bags per sq yard
    SAND_PER_SQ_YARD = 0.6 # tons per sq yard
    WATER_PER_SQ_YARD = 500 # liters per sq yard

    def __init__(self, built_up_area, floors, days=None, workers=None, daily_wage=None, cost_per_sq_yard=None):
        """
        built_up_area: in square yards
        """
```

```

WATER_PER_SQ_YARD = 500 # liters per sq yard

def __init__(self, built_up_area, floors, days=None, workers=None, daily_wage=None, cost_per_sq_yard=None):
    """
    built_up_area: in square yards
    floors: total floors (e.g., "G+3" means 4 floors)
    days: construction days (optional)
    workers: number of workers (optional)
    daily_wage: custom daily wage per worker (optional)
    cost_per_sq_yard: custom cost per sq yard
    """
    self.built_up_area = float(built_up_area)
    self.num_floors = self._parse_floors(floors)
    self.days = days
    self.workers = workers
    self.DAILY_WAGE_PER_WORKER = daily_wage or self.DAILY_WAGE_PER_WORKER
    self.COST_PER_SQ_YARD = cost_per_sq_yard or self.COST_PER_SQ_YARD

def _parse_floors(self, floors_str):
    """Parse floor format like 'G+3' to total floors"""
    if isinstance(floors_str, int):
        return floors_str
    floors_str = str(floors_str).upper().replace(" ", "")
    if "+" in floors_str:
        return int(floors_str.split("+")[1]) + 1
    return int(floors_str)
  
```

```

# =====
# ROUTES
# =====

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/api/calculate", methods=["POST"])
def api_calculate():
    """Main calculation endpoint"""
    try:
        data = request.json

        # Validate inputs
        built_up_area = float(data.get("built_up_area", 0))
        floors = data.get("floors", "G+2")
        days = int(data.get("days", 0)) if data.get("days") else None
        workers = int(data.get("workers", 0)) if data.get("workers") else None
        daily_wage = float(data.get("daily_wage", 500)) if data.get("daily_wage") else 500
        cost_per_sq_yard = float(data.get("cost_per_sq_yard", 1500)) if data.get("cost_per_sq_yard") else 1500
    
```

```
def api_calculate():
    }
}

    return jsonify(result), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

@app.route("/health")
def health():
    return jsonify({
        "status": "healthy",
        "system": "Construction Planning System",
        "timestamp": time.time()
    })

# =====
# MAIN
# =====

if __name__ == "__main__":
    print("🚧 Construction Planning System running")
    print("🔗 http://localhost:5000")
    print("❤️ Health: /health")
    app.run(host="0.0.0.0", port=5000, debug=True)
```

MILESTONE 3: Frontend Development

This milestone designs and develops a modern, responsive user interface using HTML, CSS, and JavaScript. It allows users to input project details and visually view construction plans, costs, schedules, and blueprints.

Create modern, responsive user interface

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Construction Planning System - AI-Powered Project Planning</title>
  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: #f8f9fa;
      color: #333;
    }

    /* ===== NAVBAR ===== */
    nav {
      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
      padding: 15px 0;
      position: sticky;
      top: 0;
      z-index: 1000;
      box-shadow: 0 4px 12px rgba(0,0,0,0.15);
    }
  </style>
</head>
```

```
<html lang="en">
<head>
  <style>
    .nav-container {
      display: flex;
      justify-content: space-between;
      align-items: center;
    }

    .logo {
      color: white;
      font-size: 1.8em;
      font-weight: bold;
      display: flex;
      align-items: center;
      gap: 10px;
    }

    .nav-links {
      display: flex;
      gap: 30px;
      list-style: none;
    }

    .nav-links a {
      color: white;
      text-decoration: none;
      font-weight: 600;
      transition: opacity 0.3s;
    }
  </style>
</head>
```

```
<html lang="en">
<body>
  <script>
    function displayWeeklyPlan(weeklyPlan) {

      const finalPlan = [...enhancedPlan, ...additionalPhases];

      timeline.innerHTML = finalPlan.map(item => `
        <div class="timeline-item">
          <div class="timeline-content">
            <div class="timeline-week">Week ${item.week}</div>
            <div class="timeline-phase">${item.phase}</div>
            <div class="timeline-activities">${item.activities.join(' • ')}</div>
          </div>
        </div>
      `).join('');

      weeklyPlanSection.style.display = 'block';
    }

    function displayAssumptions(assumptions, input) {
      const assumptionsSection = document.getElementById('assumptionsSection');
      const assumptionsContent = document.getElementById('assumptionsContent');

      assumptionsContent.innerHTML = `
        <div class="note-item">
          <div class="note-label">📍 Location</div>
          <div class="note-value">${assumptions.location}</div>
        </div>
      `;
    }
  </script>
</body>
</html>
```

```
<html lang="en">
<body>
  <script>
    function displayAssumptions(assumptions, input) {
      const assumptionsSection = document.getElementById('assumptionsSection');
      const assumptionsContent = document.getElementById('assumptionsContent');

      assumptionsContent.innerHTML = `
        <div class="note-item">
          <div class="note-label">📍 Location</div>
          <div class="note-value">${assumptions.location}</div>
        </div>
        <div class="note-item">
          <div class="note-label">🏠 Cost per Sq. Yard</div>
          <div class="note-value">₹${assumptions.cost_per_sq_yard} per sq. yard</div>
        </div>
        <div class="note-item">
          <div class="note-label">🏢 Overhead Percentage</div>
          <div class="note-value">${assumptions.overhead_percentage}%</div>
        </div>
        <div class="note-item">
          <div class="note-label">🏡 Built-up Area</div>
          <div class="note-value">${input.built_up_area} Sq. Yards</div>
        </div>
        <div class="note-item">
          <div class="note-label">🏠 Total Floors</div>
          <div class="note-value">${input.num_floors} (${input.floors})</div>
        </div>
      `;

      assumptionsSection.style.display = 'block';
    }
  </script>
</body>
</html>
```

MILESTONE 4: Deployment

This phase focuses on deploying the system locally with proper environment configuration and logging. It ensures that the complete application runs smoothly in a production-like setup.

Activity 4.1: Local Deployment

Run the application:

```
PS C:\Users\Surya\OneDrive\Desktop\ai construction> python app2.py
=====
Construction Planning System
Model: granite3.3:2b
Running locally via Ollama
=====
Construction Planning System running
http://localhost:5000
Health: /health
* Serving Flask app 'app2'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.68.61:5000
```

```
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.68.61:5000
Press CTRL+C to quit
* Restarting with watchdog (windowsapi)
=====
Construction Planning System
Model: granite3.3:2b
Running locally via Ollama
=====
Construction Planning System running
http://localhost:5000
Health: /health
* Debugger is active!
* Debugger PIN: 519-512-381
```

 **BuildAI Pro**

Home Calculator Features Contact



AI-Powered Construction Planning

Plan your construction projects with precision using advanced AI algorithms. Get accurate worker requirements, realistic timelines, detailed blueprints, and comprehensive cost analysis in seconds.

Start Planning Now

Learn More

 **BuildAI Pro**

Home Calculator Features Contact

Why Choose BuildAI Pro?

Comprehensive features designed for modern construction management



Smart Calculations

Advanced algorithms calculate worker requirements, timelines, and costs based on project complexity and constraints.



Professional Blueprints

Auto-generated architectural blueprints with room layouts, dimensions, and detailed floor plans for every project.



Detailed Schedules

Comprehensive week-by-week construction schedules covering all phases from site prep to final handover.



Cost Analysis

Transparent cost breakdown including labor, materials, overhead with customizable regional rates.



Workforce Planning

Intelligent worker allocation across 5 categories: masons, helpers, steel workers, carpenters, supervisors.



Real-Time Processing

Instant calculations and results with AI-powered insights for better decision making.



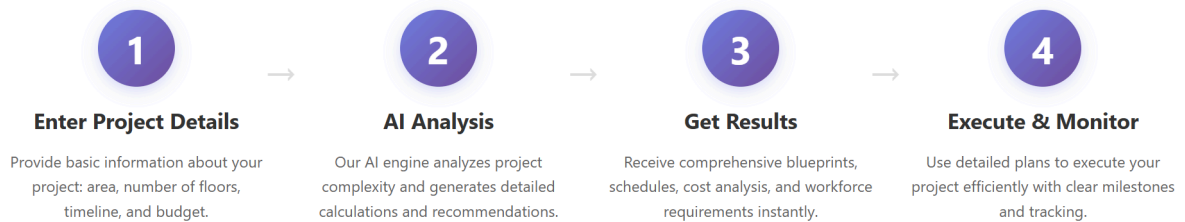
Transform Your Construction Planning

- ✓ **Save Time:** Get comprehensive project plans in seconds instead of days
- ✓ **Reduce Costs:** Accurate estimations prevent budget overruns and wastage
- ✓ **Better Planning:** Detailed blueprints and schedules ensure smooth execution
- ✓ **Workforce Optimization:** Right number of workers at the right time
- ✓ **Risk Mitigation:** Built-in buffers for weather, curing, and delays
- ✓ **Regional Accuracy:** Customizable wage rates and material costs for





How BuildAI Pro Works



Powered by Advanced Technology

**IBM Granite 3.3
2B**AI-powered insights and
analysis**Flask Framework**

Robust backend infrastructure

**Advanced
Algorithms**

Precise calculation engine

**Responsive
Design**

Works on all devices

MILESTONE 5: Integration & Testing

This milestone integrates frontend, backend, and AI components and performs functional and performance testing. It validates calculation accuracy, system reliability, and overall user experience.

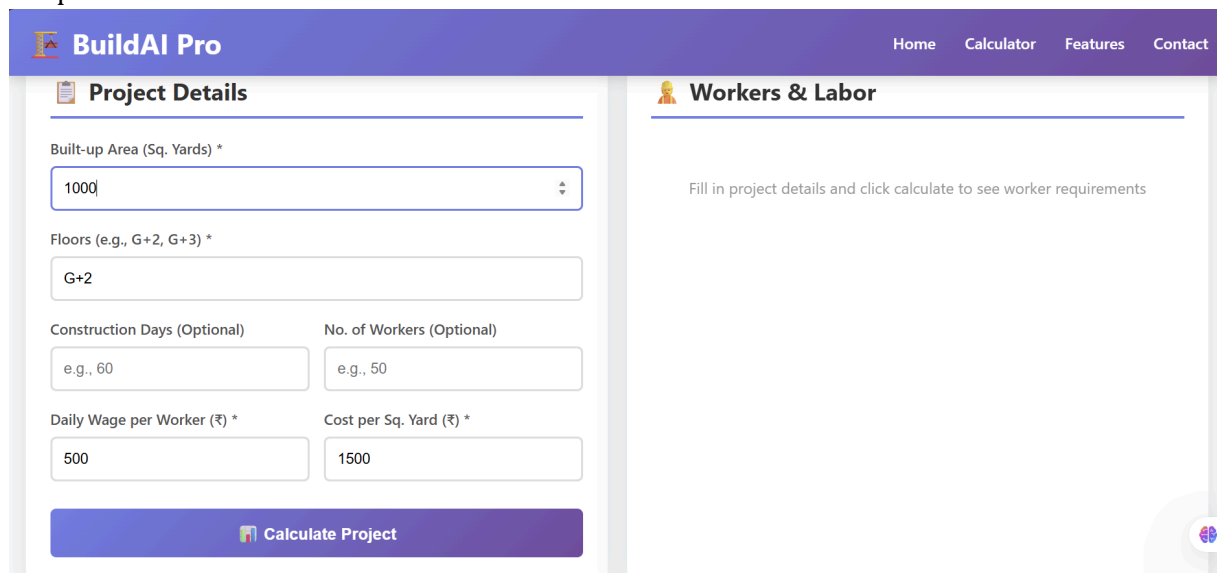
Activity 5.1: Functional Testing

Test Case :

Input:

- Built-up Area: 1000 sq yards
- Floors: G+2
- Default wage and costs

Output:



BuildAI Pro Home Calculator Features Contact

Project Details

Built-up Area (Sq. Yards) *

1000

Floors (e.g., G+2, G+3) *

G+2

Construction Days (Optional) No. of Workers (Optional)

e.g., 60 e.g., 50

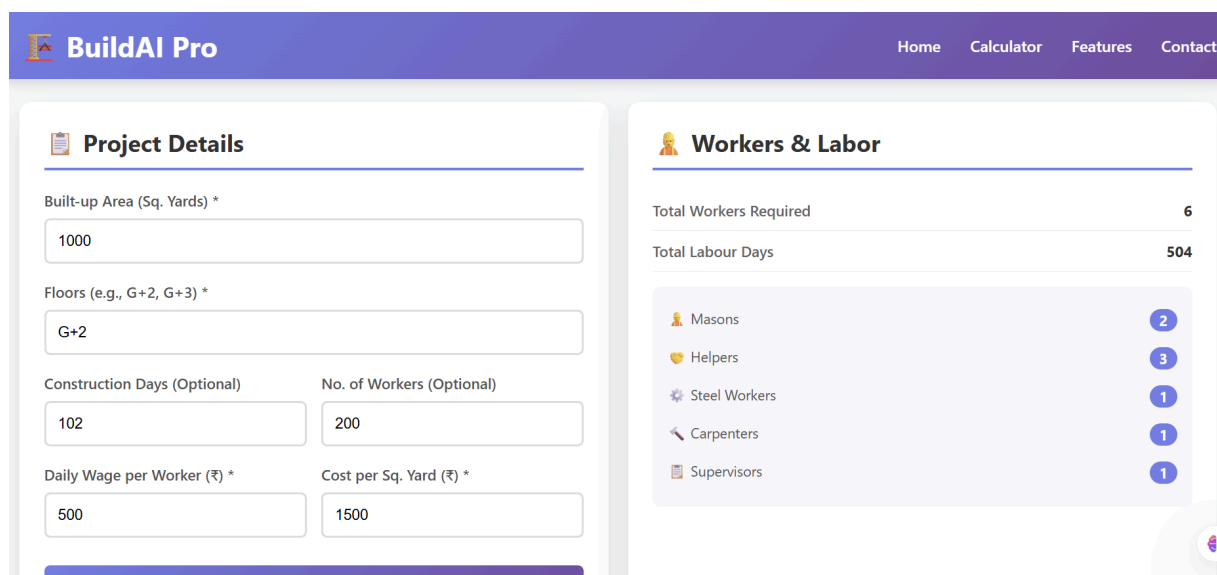
Daily Wage per Worker (₹) * Cost per Sq. Yard (₹) *

500 1500

Calculate Project

Workers & Labor

Fill in project details and click calculate to see worker requirements



BuildAI Pro Home Calculator Features Contact

Project Details

Built-up Area (Sq. Yards) *

1000

Floors (e.g., G+2, G+3) *

G+2

Construction Days (Optional) No. of Workers (Optional)

102 200

Daily Wage per Worker (₹) * Cost per Sq. Yard (₹) *

500 1500

Workers & Labor

Worker Type	Count
Total Workers Required	6
Total Labour Days	504
Masons	2
Helpers	3
Steel Workers	1
Carpenters	1
Supervisors	1

Timeline	Cost Breakdown	Materials Required
Duration (Days)84	Labor Cost₹2,52,000	Steel10.5 tons
Duration (Weeks)12	Material Cost₹15,00,000	Cement Bags1200 bags
Duration (Months)3	Overhead (10%)₹1,75,200	Sand1800 tons
	Total Cost₹19,27,200	Water15,00,000 L

BuildAI Pro

HomeCalculatorFeaturesContact

Architectural Blueprint

FLOOR 3

MASTER BEDROOM

Bed 1

10.5' x 15.8'

BEDROOM 2

Bed 2

10.5' x 15.8'

LIVING ROOM

Hall

12.6' x 19.0'

KITCHEN

KT

8.4' x 12.6'

BATHROOM

WC

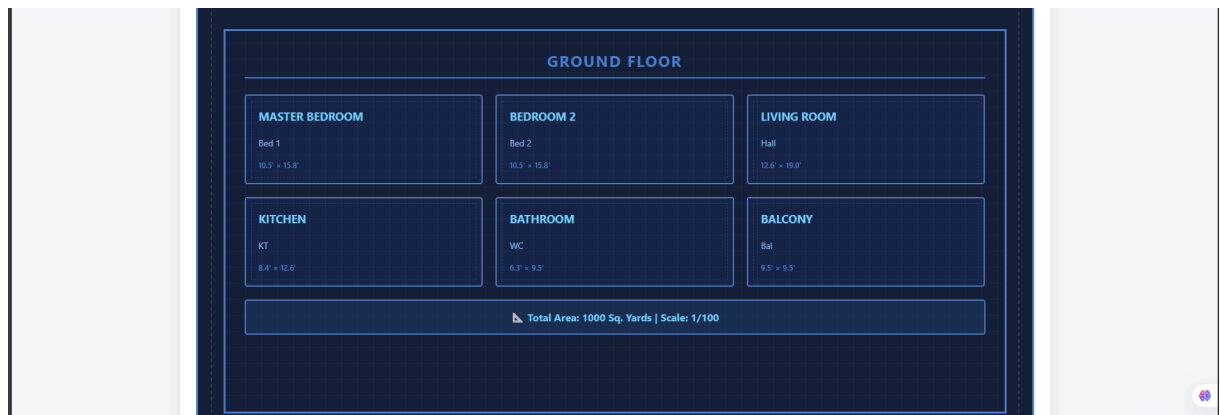
6.3' x 9.5'

BALCONY

Bal

9.5' x 9.5'

Total Area: 1000 Sq. Yards | Scale: 1/100



Construction Schedule



Week 1

Site Preparation

Site clearing • Leveling • Setting up temporary facilities



Week 2

Foundation Work

Excavation • Foundation laying • Concrete curing



Week 3

Ground Floor Slab

Formwork preparation • Reinforcement placement • Concrete pouring • Curing



Week 5

Floor 2 Slab

Formwork preparation • Reinforcement placement • Concrete pouring • Curing



Week 7

Floor 3 Slab

Formwork preparation • Reinforcement placement • Concrete pouring • Curing



Week 9

Brickwork & Masonry

Brick laying • Wall plastering prep



Week 12

Plastering

Wall plastering • Ceiling plastering



Week 14

Electrical Fittings & Installation

Light fixture installation • Switch board setup • Wiring completion • Safety check



Week 16

Plumbing Fixtures & Sanitary

Bathroom fixtures • Kitchen fittings • Water system testing • Drainage check



Week 18

Wall Painting & Color Work

Primer coating • Base paint • Color application • Finishing



Week 20

Doors, Windows & Hardware

Door installation • Window fitting • Hardware mounting • Glass fitting



Week 22

Final Finishing & Cleanup

Interior polishing • Glass cleaning • Final inspection • Handover

Assumptions & Input Parameters

 Location
India

 Building Type
Residential RCC

 Daily Wage per Worker
₹500 per day

 Cost per Sq. Yard
₹1500 per sq. yard

 Overhead Percentage
10%

 Built-up Area
1000 Sq. Yards

 Total Floors
3 (G+2)

BuildAI Pro - Construction Planning System

AI-Powered intelligent planning for modern construction projects

Powered by IBM Granite 3.3.2B | Built with Flask & Advanced Algorithms

© 2024 BuildAI Pro. All rights reserved.

Conclusion:

Construction Planning System successfully delivers an intelligent construction project planning assistant that combines advanced calculation algorithms with IBM Granite 3.3 2B AI model. By leveraging local Ollama deployment, the system provides fast, accurate, and personalized construction recommendations without requiring cloud API keys. The platform demonstrates the viability of AI-powered construction planning, making professional project estimation accessible to contractors, developers, and project managers while maintaining high performance, accuracy, and data privacy through local processing.